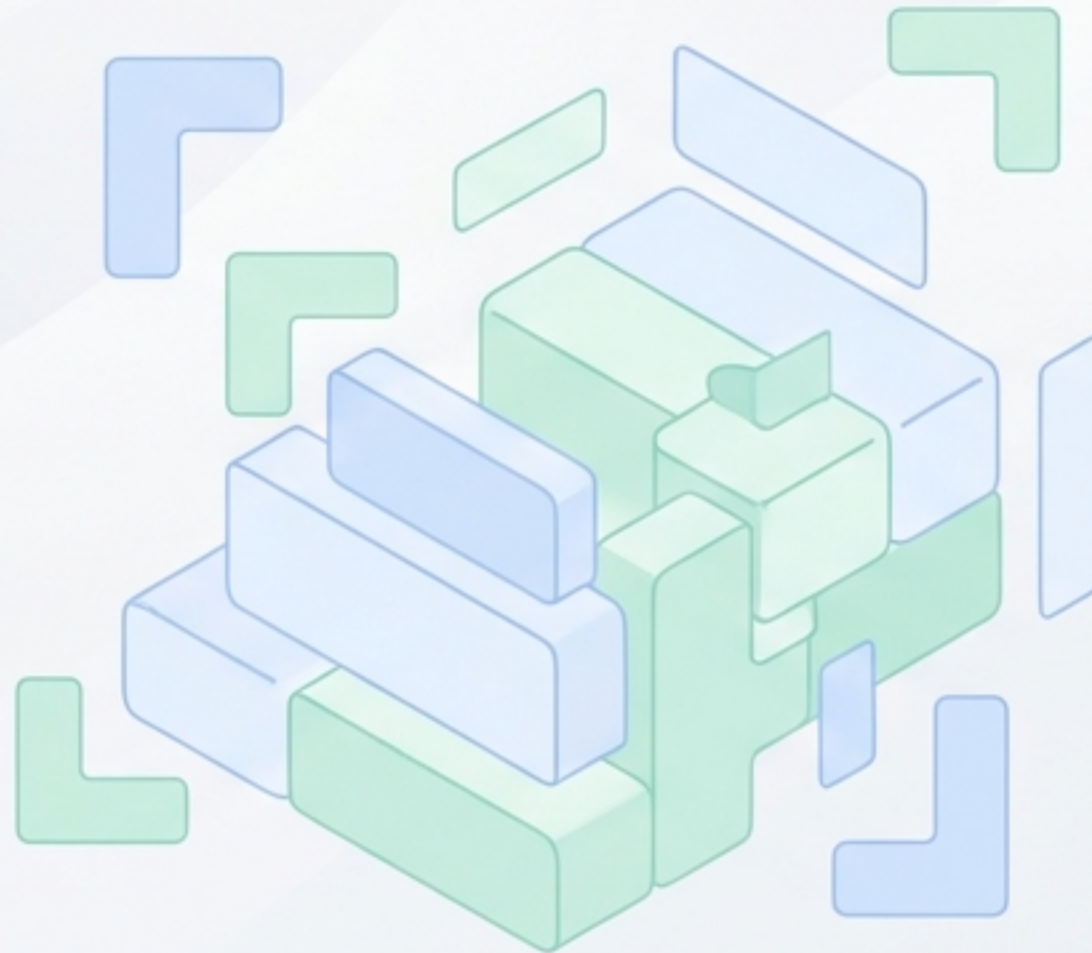


# Arquitetura de Código: Evolução e Modularização

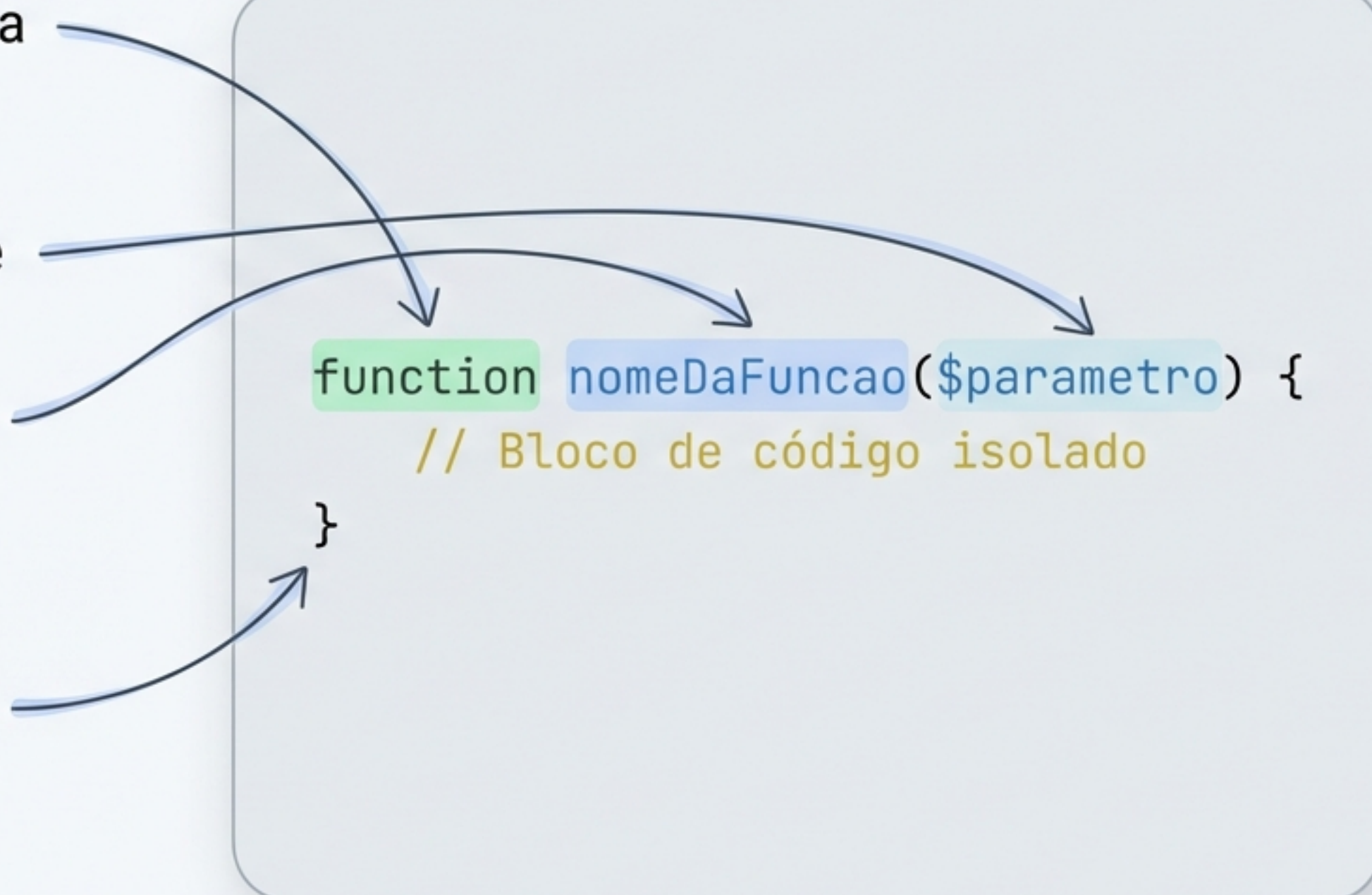
Um guia prático de desenvolvimento orientado a projetos





# Estruturando Blocos de Execução com Funções

- ✍ • **Palavra Reservada function:** Indica a criação de uma sub-rotina reaproveitável.
- ✍ • **Identificador:** O nome do bloco de código (ex: `exibeMensagem`).
- ⇄ • **Parâmetros ():** Variáveis isoladas que a função espera receber para trabalhar.
- { } • **Escopo {}:** Onde a lógica real acontece. Variáveis criadas aqui dentro não interferem no resto do programa.



```
function nomeDaFuncao($parametro) {  
    // Bloco de código isolado  
}
```



# Encapsulando as Regras de Negócio do Sistema

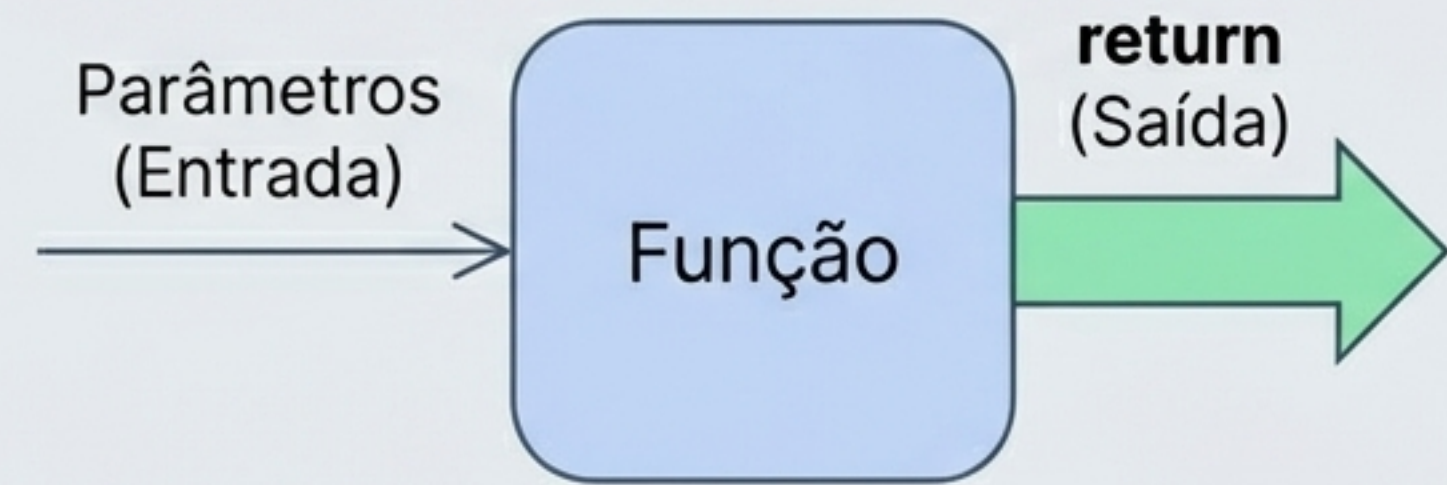
- 📁 • **O Problema:** Exibir mensagens sobre o ano de lançamento na página de catálogo e na página de detalhes gera código duplicado.
- ⚙️ • **A Solução:** Extrair a lógica do if/else para dentro da nossa própria função `exibeMensagemLancamento()`.
- ➡️ • **O Parâmetro:** Passamos a variável `$ano` para que a função seja genérica e atenda a qualquer filme.

```
function exibeMensagemLancamento($ano) {  
    if ($ano > 2022) {  
        echo "Esse filme é um lançamento\n";  
    } elseif ($ano > 2020 && $ano <= 2022) {  
        echo "Esse filme ainda é novo\n";  
    } else {  
        echo "Esse filme não é um lançamento\n";  
    }  
}
```



# Extraindo Resultados de Sub-rotinas

- ✍ • A **Palavra Reservada** **return**: Finaliza a execução da função e devolve um valor.
- ✍ • **Substituição Dinâmica**: No momento em que o código roda, a chamada da função é literalmente substituída pelo valor que ela retornou.
- { } • **Função vs. Procedimento**: Enquanto algumas funções apenas executam ações (como exibir textos), outras processam dados que precisamos armazenar em variáveis.



```
$variavel = funcao();
```



# Validando Regras Dinâmicas de Acesso

-  **Regra de Negócio:** Um filme está disponível se o usuário tem o Plano Prime OU se o filme foi lançado antes de 2020.
- ✓ **Aplicação Prática:** A função `incluidoNoPlano()` não usa `echo`. Ela usa `return` para devolver `true` ou `false`.
- ✓ **Reaproveitamento:** Podemos usar esse retorno direto em um `if` para decidir se mostramos o botão de Play no front-end.

```
function incluidoNoPlano($planoPrime,  
$anoLancamento) {  
    return $planoPrime ||  
    $anoLancamento < 2020;  
}
```



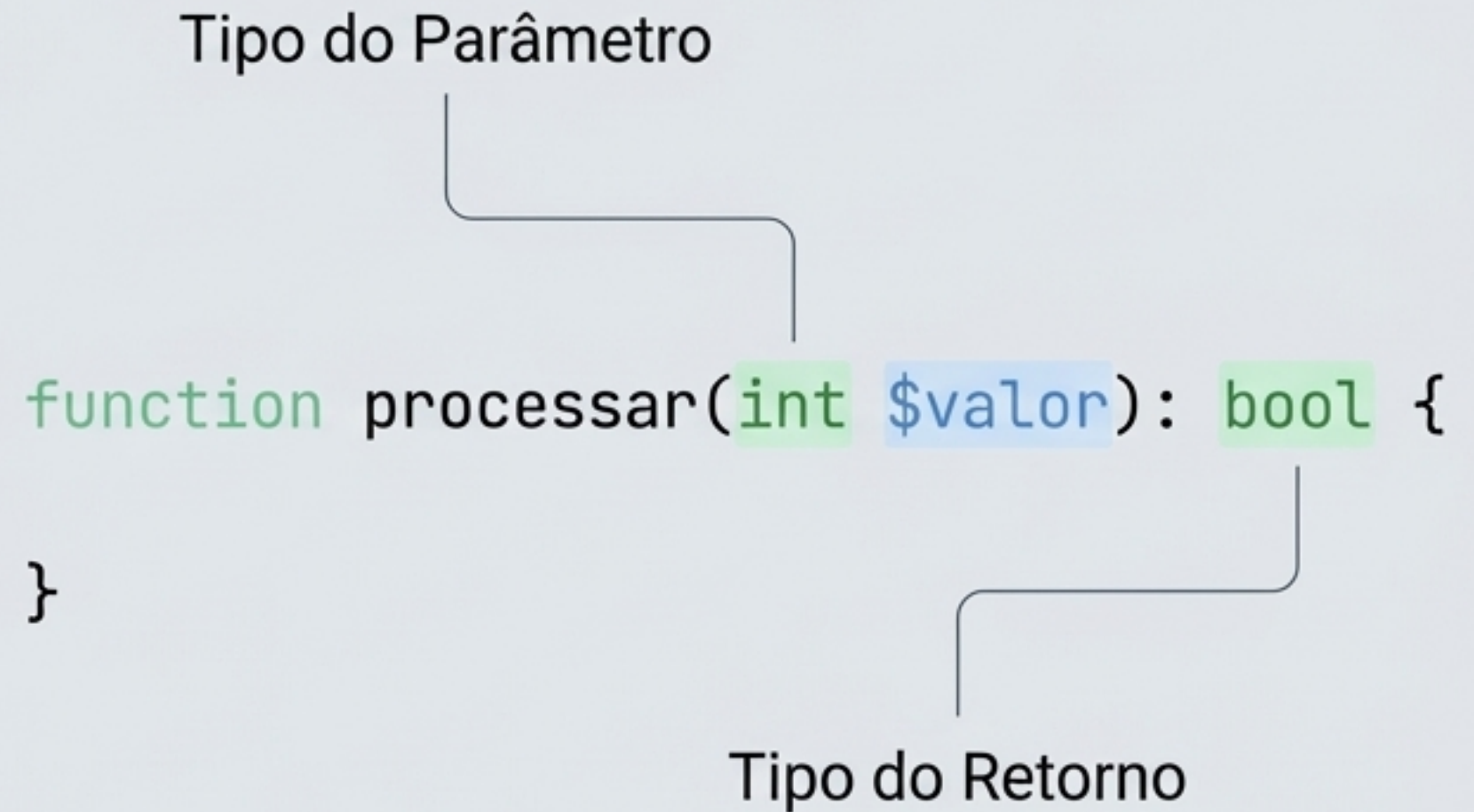
# Previsibilidade e Assinatura de Código

- **Tipagem de Parâmetros:** Declarar o tipo exato antes da variável (ex: `int`, `bool`, `string`).
- **Tipagem de Retorno:** Declarar o que a função devolve adicionando `:tipo` após os parênteses.
- **O Tipo `void`:** Indica um vácuo. Usado quando a função executa uma ação mas não possui a palavra `return`.
- **Defesa Antecipada:** O PHP avisa sobre erros antes mesmo do código rodar completamente.

Tipo do Parâmetro

```
function processar(int $valor): bool {  
  
}
```

Tipo do Retorno





# Blindando a Aplicação Contra Erros de Tipagem

- Evoluindo o Código: Reescrevemos as funções do sistema para exigir tipos específicos.
- Evitando Acidentes: Garantimos que ninguém chame `incluidoNoPlano` passando uma string no lugar de um booleano.
- Profissionalização: IDEs e editores de texto agora conseguem nos alertar sobre falhas em tempo real durante o desenvolvimento.

```
function incluidoNoPlano(bool $planoPrime, int
$anoLancamento): bool {
    return $planoPrime || $anoLancamento < 2020;
}

function exibeMensagemLancamento(int $ano):
void {
    if ($ano > 2022) {
        echo "Esse filme é um lançamento\n";
    } elseif ($ano > 2020 && $ano <= 2022) {
        echo "Esse filme ainda é novo\n";
    } else {
        echo "Esse filme não é um lançamento\n";
    }
}
```



# Operações Integradas em Coleções de Dados

## var\_dump()



Inspecionando Variáveis: A função `var_dump()` revela o tipo e o conteúdo detalhado de um array, essencial para investigar bugs.

## sort()



Ordenação: A função `sort()` organiza os itens.

Passagem por Referência (&): O `sort()` altera o array original diretamente na memória, em vez de criar uma cópia.

## min()



Busca Rápida: A função `min()` varre a coleção e encontra o menor valor.



# Processamento Analítico de Notas e Avaliações

- **O Array de Notas:** O filme recebe várias avaliações de usuários armazenadas em `$notas = [10, 7, 8.5, 5, 9.3]`.
- **Investigação Visual:** Usamos `var_dump($notas)` no terminal para enxergar a estrutura dos dados flutuantes (float).
- **Extraíndo Inteligência:** Aplicamos `sort($notas)` para organizar o ranking e `$menorNota = min($notas)` para identificar a pior avaliação do filme.

```
$notas = [10, 7, 8.5, 5, 9.3];  
var_dump($notas); // Exibe a estrutura  
sort($notas); // Ordena o array  
$menorNota = min($notas); // Encontra o  
menor valor  
echo $menorNota; // Exibe 5
```

COPIAR CÓDIGO

float(5)



# Tratamento e Extração de Textos em Strings

- Mapeamento de Posição: A função `strpos($string, $busca)` localiza em qual índice numérico um caractere específico aparece pela primeira vez (lembrando que a contagem inicia em 0).
- Recorte Direcionado: A função `substr($string, $inicio, $tamanho)` extrai um pedaço exato do texto com base em coordenadas.
- Parâmetros Opcionais: Algumas funções nativas possuem parâmetros que assumem valores padrão se não forem preenchidos.



```
echo strpos("AVATAR", "V"); // Saída: 1
```



# Higienização e Recorte de Títulos de Filmes

- **O Desafio Front-End:** Filmes como Thor: Ragnarok possuem um título principal e um subtítulo. Queremos exibir apenas a franquia principal em destaque.
- **Mapeando o Separador:** Usamos `strpos($filme['nome'], ':')` para descobrir matematicamente onde o caractere de dois-pontos está.
- **O Recorte:** Alimentamos essa posição no `substr()` para extrair e isolar apenas a palavra Thor.

Thor: Ragnarok

```
$filme = ['nome' => 'Thor: Ragnarok'];  
$posicaoDoisPontos = strpos($filme['nome'], ':');  
$tituloPrincipal = substr($filme['nome'], 0,  
$posicaoDoisPontos);  
var_dump($tituloPrincipal);
```

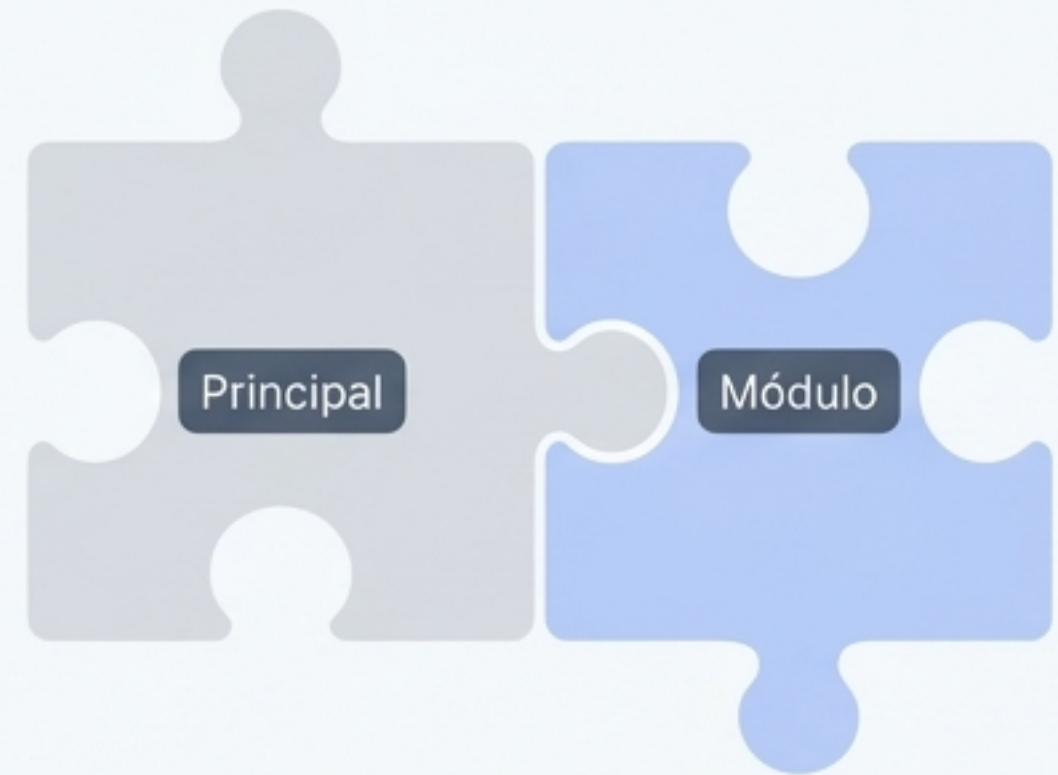
string(4) "Thor"

COPIAR CÓDIGO



# Inclusão Segura de Módulos Externos

- A Palavra Reservada `require`: Puxa o conteúdo de outro arquivo e o injeta no script atual. Se o arquivo não existir, o sistema para (fatal error).
- Caminhos Relativos vs. Absolutos: Dependendo da pasta de onde o script é executado pode gerar erros em servidores reais.
- A Constante Mágica `__DIR__`: Retorna dinamicamente o caminho absoluto e completo do diretório do arquivo atual, tornando a inclusão à prova de falhas de navegação.

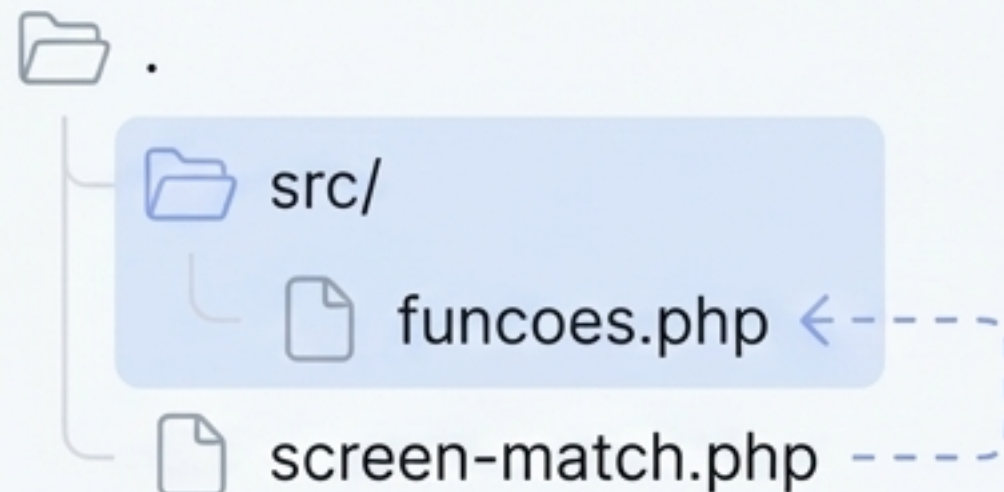


```
require __DIR__ . '/caminho';
```



# Desacoplando a Lógica em Múltiplos Arquivos

- **O Problema do Monolito:** Nosso script chegou a mais de 70 linhas misturando regras de negócio, dados e execução.
- **A Separação Física:** Extraímos as definições das nossas funções para um arquivo isolado chamado `funcoes.php`.
- **A Integração:** O código fica limpo, legível e organizado como nos projetos de grandes equipes.



```
<?php

require __DIR__ . '/src/funcoes.php';

// # código omitido
```

COPIAR CÓDIGO



# Código Limpo, Profissional e Pronto para Escalar

Hoje, saímos do roteiro básico para a mentalidade de engenharia de software.

**Isolamos comportamentos.**

Garantimos a **segurança** dos dados com **tipagem**.

**Modularizamos** a arquitetura.

**Próximo passo:** dominar o consumo e fornecimento desses dados estruturados.